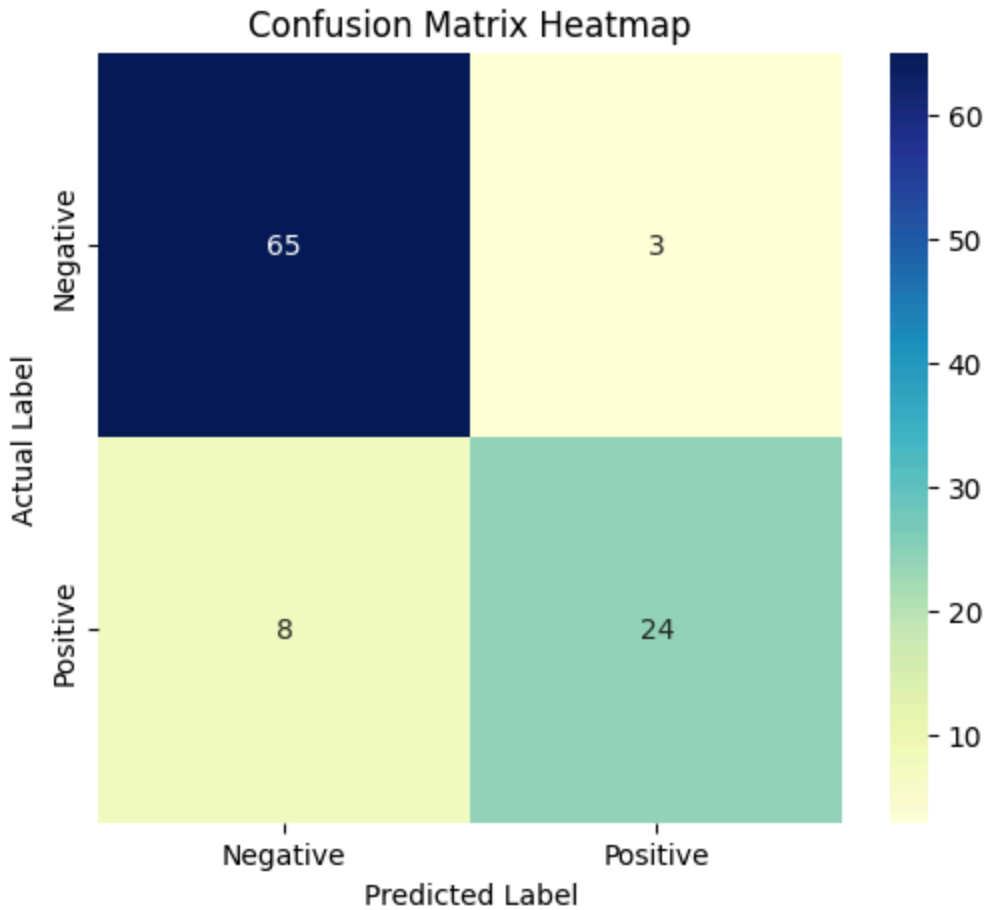
Confusion Matrix

In [1]:
```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# Confusion matrix values
TP = 24
TN = 65
FP = 3
FN = 8
N = TP + TN + FP + FN  # Total samples

# Create confusion matrix
conf_matrix = np.array([[TN, FP],
                        [FN, TP]])

# Labels
labels = ['Negative', 'Positive']

# Plot heatmap
plt.figure(figsize=(6, 5))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='YlGnBu',
            xticklabels=labels, yticklabels=labels)
plt.xlabel('Predicted Label')
plt.ylabel('Actual Label')
plt.title('Confusion Matrix Heatmap')
plt.show()

# Calculate metrics
accuracy = (TP + TN) / N
precision = TP / (TP + FP)
recall = TP / (TP + FN)  # Sensitivity
f1_score = 2 * (precision * recall) / (precision + recall)
specificity = TN / (TN + FP)
negative_predictive_value = TN / (TN + FN)

# Print metrics
print("Accuracy: ",accuracy)
print("Precision: ",precision)
print("Recall (Sensitivity): ",recall)
print("F1 Score:",f1_score)
print("Specificity: ",specificity)
print("Negative Predictive Value: ",negative_predictive_value)
```




```
Accuracy:  0.89
Precision:  0.8888888888888888
Recall (Sensitivity):  0.75
F1 Score: 0.8135593220338982
Specificity:  0.9558823529411765
Negative Predictive Value:  0.8904109589041096
```

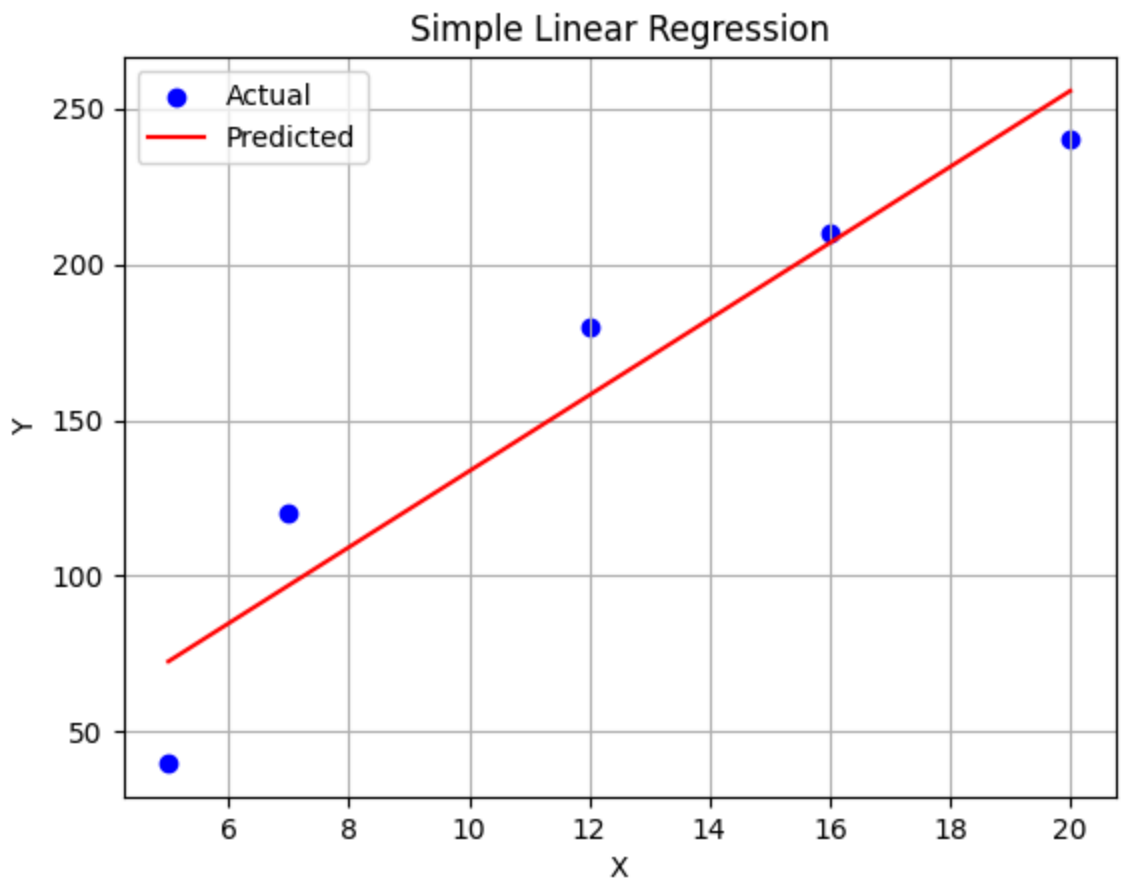
Simple linear Regression(Using Library function)

In [4]:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Sample data
x = np.array([5, 7, 12, 16, 20]).reshape(-1, 1)  # Independent variable
y = np.array([40, 120, 180, 210, 240])          # Dependent variable

# Create and train the model
model = LinearRegression()
model.fit(x, y)
```

```python
# Predict values
y_pred = model.predict(x)

# Print model parameters
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)

# Calculate error metrics
mse = mean_squared_error(y, y_pred)
mae = mean_absolute_error(y, y_pred)
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Plot actual vs predicted
plt.scatter(x, y, color='blue', label='Actual')
plt.plot(x, y_pred, color='red', label='Predicted')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Simple Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

```
Slope (Coefficient): 12.20779220779221
Intercept: 11.506493506493484
Mean Squared Error (MSE): 465.87012987012986
Mean Absolute Error (MAE): 19.283116883116882
Root Mean Squared Error (RMSE): 21.58402487651758
```

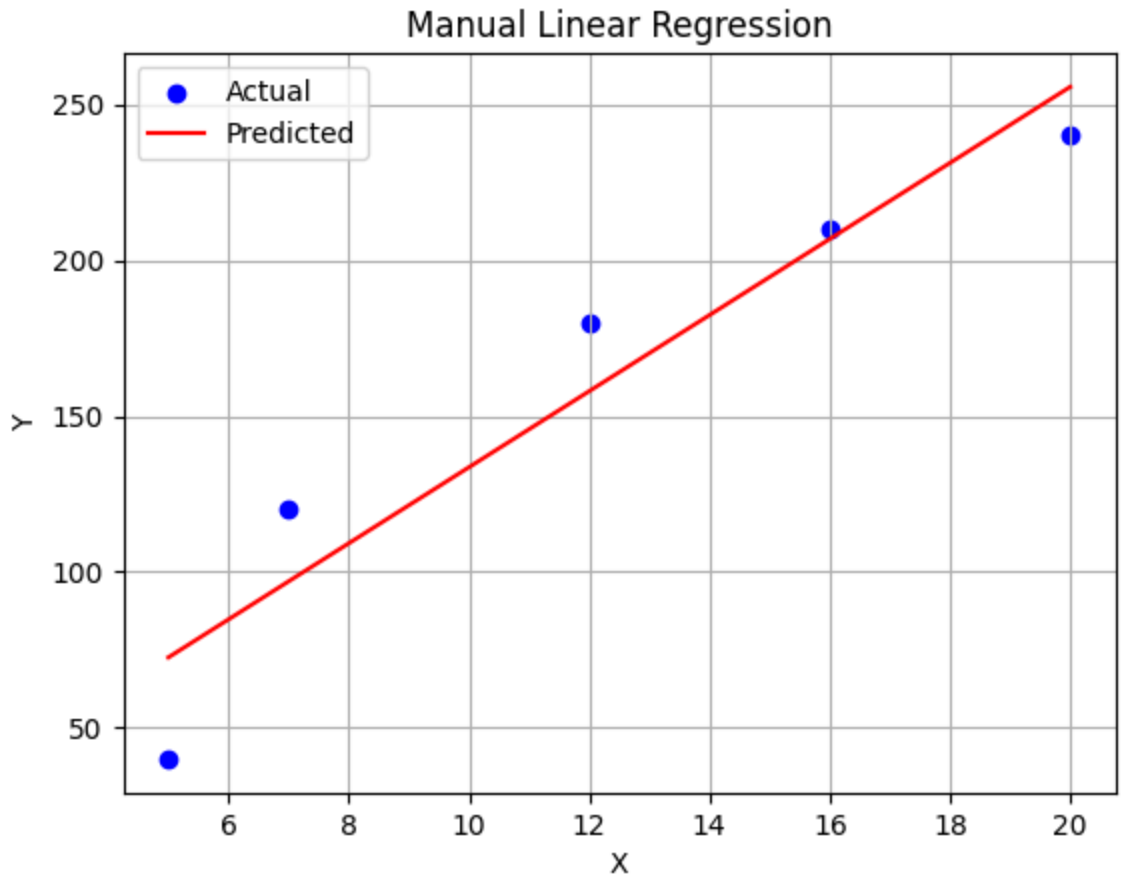


simple linear function without Library function

In [5]:
```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.array([5, 7, 12, 16, 20])
y = np.array([40, 120, 180, 210, 240])

# Step 1: Calculate slope (m) and intercept (b) manually
n = len(x)
x_mean = np.mean(x)
y_mean = np.mean(y)

# Calculate slope (m)
numerator = np.sum((x - x_mean) * (y - y_mean))
denominator = np.sum((x - x_mean) ** 2)
slope = numerator / denominator

# Calculate intercept (b)
intercept = y_mean - slope * x_mean
```

```python
print("Slope (Coefficient):", slope)
print("Intercept:", intercept)

# Step 2: Predict values
y_pred = slope * x + intercept

# Step 3: Calculate error metrics manually
mse = np.mean((y - y_pred) ** 2)
mae = np.mean(np.abs(y - y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 4: Plot actual vs predicted
plt.scatter(x, y, color='blue', label='Actual')
plt.plot(x, y_pred, color='red', label='Predicted')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Manual Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

```
Slope (Coefficient): 12.207792207792208
Intercept: 11.506493506493513
Mean Squared Error (MSE): 465.87012987013014
Mean Absolute Error (MAE): 19.283116883116886
Root Mean Squared Error (RMSE): 21.58402487651759
```

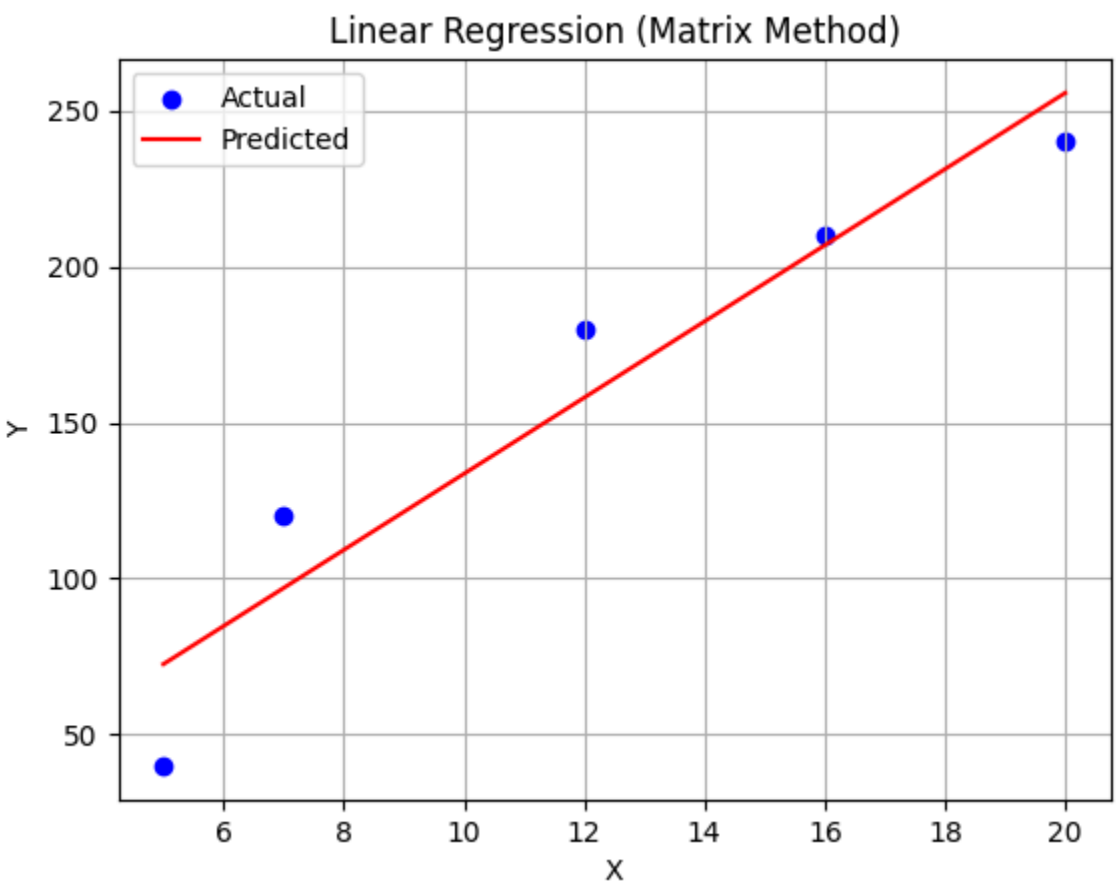


Simple Linear Regression(using Matrix method)

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
x = np.array([5, 7, 12, 16, 20])
y = np.array([40, 120, 180, 210, 240])

# Step 1: Prepare the design matrix X and target vector Y
X = np.vstack([np.ones(len(x)), x]).T  # Add intercept term
Y = y.reshape(-1, 1)

# Step 2: Compute coefficients using the Normal Equation: θ = (XᵀX)⁻¹XᵀY
theta = np.linalg.inv(X.T @ X) @ X.T @ Y
intercept, slope = theta.flatten()

print("Slope (Coefficient):", slope)
print("Intercept:", intercept)

# Step 3: Predict values
y_pred = X @ theta
```

```python
y_pred = y_pred.flatten()

# Step 4: Calculate error metrics
mse = np.mean((y - y_pred) ** 2)
mae = np.mean(np.abs(y - y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 5: Plot actual vs predicted
plt.scatter(x, y, color='blue', label='Actual')
plt.plot(x, y_pred, color='red', label='Predicted')
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Linear Regression (Matrix Method)')
plt.legend()
plt.grid(True)
plt.show()
```

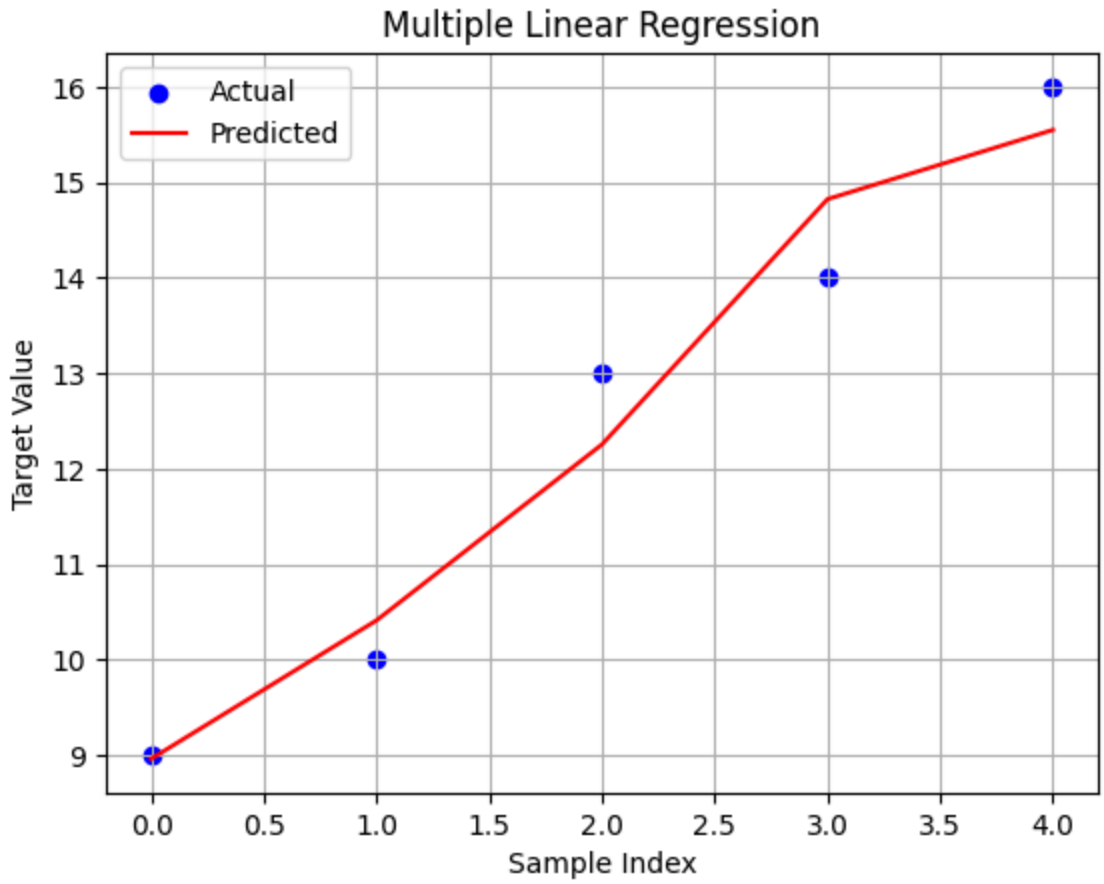
```
Slope (Coefficient): 12.207792207792213
Intercept: 11.506493506493598
Mean Squared Error (MSE): 465.8701298701296
Mean Absolute Error (MAE): 19.283116883116847
Root Mean Squared Error (RMSE): 21.584024876517578
```



Multiple linear Regression(Using Library function)

In [7]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Sample data: Two independent variables (X1 and X2)
# Shape: (number of samples, number of features)
X = np.array([
    [1, 10],
    [3, 14],
    [4, 15],
    [6, 18],
    [7, 20]
])

# Dependent variable
y = np.array([9, 10, 13, 14, 16])

# Create and train the model
model = LinearRegression()
model.fit(X, y)

# Predict values
y_pred = model.predict(X)

# Print model parameters
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)

# Optional: Plot actual vs predicted
plt.scatter(range(len(y)), y, color='blue', label='Actual')
plt.plot(range(len(y_pred)), y_pred, color='red', label='Predicted')
plt.xlabel('Sample Index')
plt.ylabel('Target Value')
plt.title('Multiple Linear Regression')
plt.legend()
plt.grid(True)
plt.show()
```

```
Coefficients: [ 2.96078431 -1.11764706]
Intercept: 17.17647058823531
```

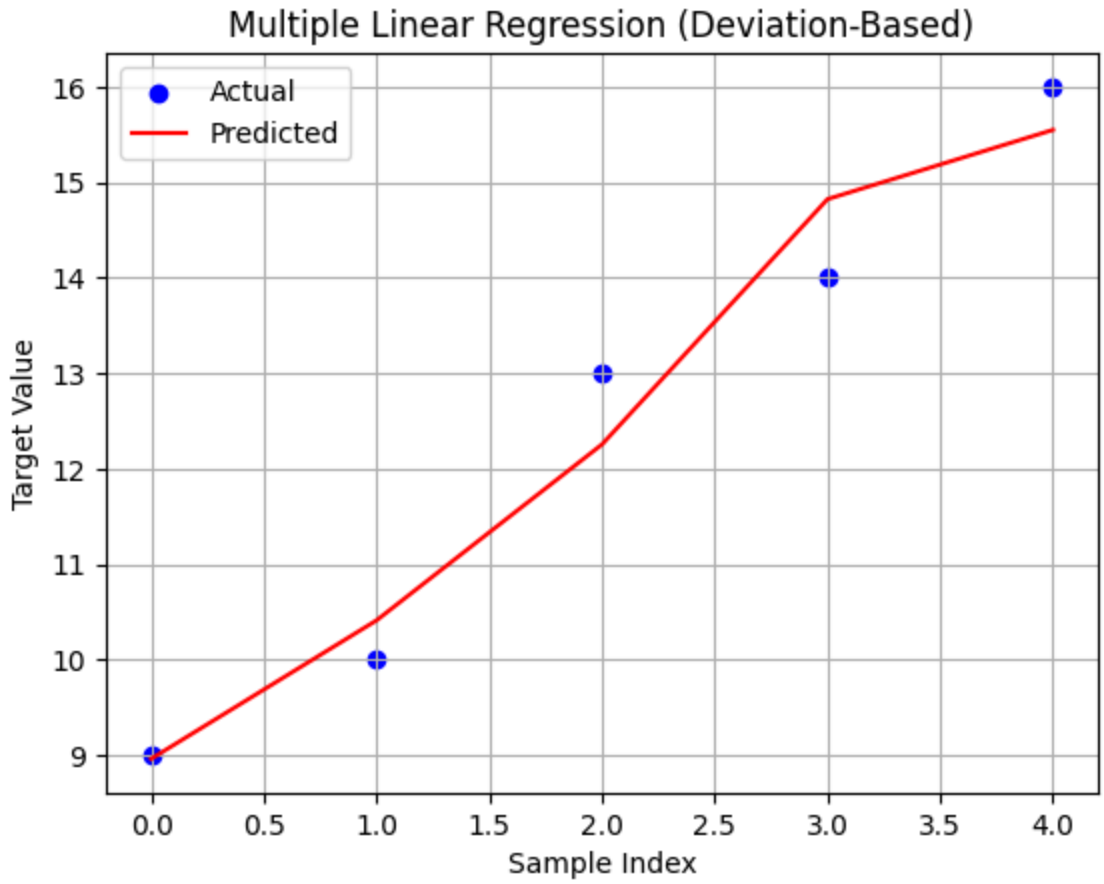


multiple linear regression (without library function)

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt

# Sample data
X1 = np.array([1, 3, 4, 6, 7])
X2 = np.array([10, 14, 15, 18, 20])
Y = np.array([9, 10, 13, 14, 16])
n = len(Y)

# Step 1: Compute required summations
sum_X1 = np.sum(X1)
sum_X2 = np.sum(X2)
sum_Y = np.sum(Y)

sum_X1_squared = np.sum(X1 ** 2)
sum_X2_squared = np.sum(X2 ** 2)
sum_X1X2 = np.sum(X1 * X2)
sum_X1Y = np.sum(X1 * Y)
sum_X2Y = np.sum(X2 * Y)
```

```python
# Step 2: Compute deviation-based terms
S_x1_squared = sum_X1_squared - (sum_X1 ** 2) / n
S_x2_squared = sum_X2_squared - (sum_X2 ** 2) / n
S_x1x2 = sum_X1X2 - (sum_X1 * sum_X2) / n
S_x1y = sum_X1Y - (sum_X1 * sum_Y) / n
S_x2y = sum_X2Y - (sum_X2 * sum_Y) / n

# Step 3: Compute coefficients using your formulas
denominator = (S_x1_squared * S_x2_squared) - (S_x1x2 ** 2)

beta_1 = ((S_x2_squared * S_x1y) - (S_x1x2 * S_x2y)) / denominator
beta_2 = ((S_x1_squared * S_x2y) - (S_x1x2 * S_x1y)) / denominator

# Step 4: Compute intercept
X1_mean = np.mean(X1)
X2_mean = np.mean(X2)
Y_mean = np.mean(Y)

beta_0 = Y_mean - beta_1 * X1_mean - beta_2 * X2_mean

print("Intercept (β₀):", beta_0)
print("Coefficient β₁ (for X₁):", beta_1)
print("Coefficient β₂ (for X₂):", beta_2)

# Step 5: Predict values
Y_pred = beta_0 + beta_1 * X1 + beta_2 * X2

# Step 6: Error metrics
mse = np.mean((Y - Y_pred) ** 2)
mae = np.mean(np.abs(Y - Y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 7: Plot actual vs predicted
plt.scatter(range(n), Y, color='blue', label='Actual')
plt.plot(range(n), Y_pred, color='red', label='Predicted')
plt.xlabel('Sample Index')
plt.ylabel('Target Value')
plt.title('Multiple Linear Regression (Deviation-Based)')
plt.legend()
plt.grid(True)
plt.show()
```

```
Intercept (β₀): 17.176470588236597
Coefficient β₁ (for X₁): 2.960784313725723
Coefficient β₂ (for X₂): -1.1176470588236775
Mean Squared Error (MSE): 0.32156862745097836
Mean Absolute Error (MAE): 0.4941176470588065
Root Mean Squared Error (RMSE): 0.5670702138633085
```



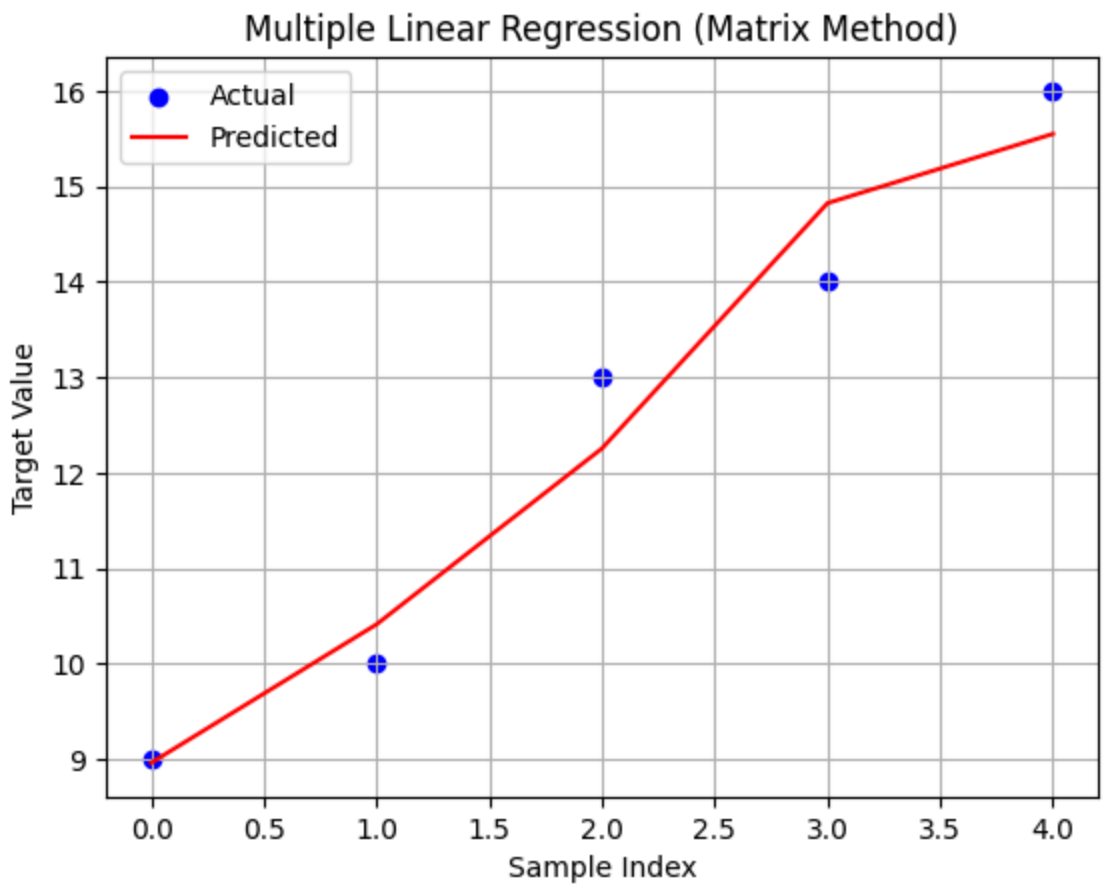
multiple linear regression (matrix method)

In [2]:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample data: Two independent variables (X1 and X2)
X = np.array([
    [1, 10],
    [3, 14],
    [4, 15],
    [6, 18],
    [7, 20]
])

# Dependent variable
y = np.array([9, 10, 13, 14, 16])

# Step 1: Add intercept term (column of ones)
X_design = np.hstack((np.ones((X.shape[0], 1)), X))  # Shape: (n_samples, n_fe

# Step 2: Compute coefficients using Normal Equation: θ = (XᵀX)⁻¹Xᵀy
theta = np.linalg.inv(X_design.T @ X_design) @ X_design.T @ y
```

```python
# Extract intercept and coefficients
intercept = theta[0]
coefficients = theta[1:]

print("Intercept:", intercept)
print("Coefficients:", coefficients)

# Step 3: Predict values
y_pred = X_design @ theta

# Step 4: Calculate error metrics manually
mse = np.mean((y - y_pred) ** 2)
mae = np.mean(np.abs(y - y_pred))
rmse = np.sqrt(mse)

print("Mean Squared Error (MSE):", mse)
print("Mean Absolute Error (MAE):", mae)
print("Root Mean Squared Error (RMSE):", rmse)

# Step 5: Plot actual vs predicted
plt.scatter(range(len(y)), y, color='blue', label='Actual')
plt.plot(range(len(y_pred)), y_pred, color='red', label='Predicted')
plt.xlabel('Sample Index')
plt.ylabel('Target Value')
plt.title('Multiple Linear Regression (Matrix Method)')
plt.legend()
plt.grid(True)
plt.show()
```

```
Intercept: 17.176470588235418
Coefficients: [ 2.96078431 -1.11764706]
Mean Squared Error (MSE): 0.32156862745098036
Mean Absolute Error (MAE): 0.4941176470587386
Root Mean Squared Error (RMSE): 0.5670702138633102
```



Polynomial (Quadratic regression model for the following data)-using matrix method

In [1]:
```
import numpy as np

# Input data
x = np.array([3, 4, 5, 6, 7])
y = np.array([2.5, 3.2, 3.8, 6.5, 11.5])
n = len(x)

# Step 1: Create the design matrix for quadratic regression
# Each row of X is [1, x_i, x_i^2]
X = np.vstack([np.ones(n), x, x**2]).T

# Step 2: Compute matrices A = XᵀX and B = Xᵀy
A = X.T @ X
B = X.T @ y

# Step 3: Define a function to compute the adjoint of a 3x3 matrix
def adjoint_3x3(M):
    adj = np.zeros((3, 3))
```

```
    for i in range(3):
        for j in range(3):
            minor = np.delete(np.delete(M, i, axis=0), j, axis=1)
            cofactor = ((-1) ** (i + j)) * np.linalg.det(minor)
            adj[j, i] = cofactor  # Transpose while assigning
    return adj

# Step 4: Compute the inverse of A using its adjoint and determinant
det_A = np.linalg.det(A)
adj_A = adjoint_3x3(A)
A_inv = adj_A / det_A

# Step 5: Solve for regression coefficients: alpha = A⁻¹ * B
alpha = A_inv @ B
a0, a1, a2 = alpha

#Print the full equation
print("y=", a2 ,"x² + ",a1 ,"x + ",a0)
```

```
y= 0.764285714285887 x² +  -5.512857142857804 x +  12.42857142858098
```

Polynomial (Quadratic regression model for the following data)-using matrix method using library function

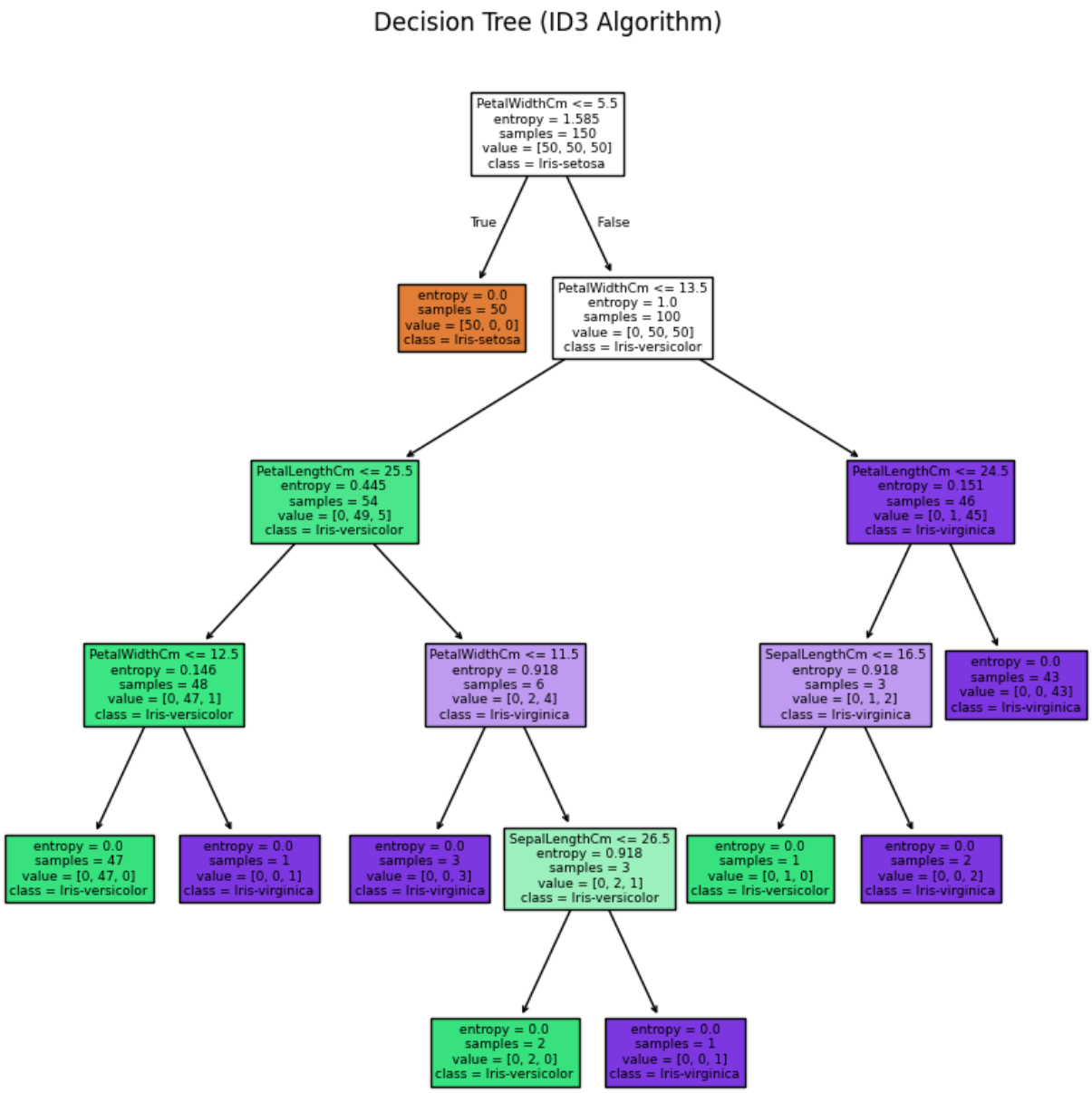
In [6]:
```
import numpy as np

# Data
x = np.array([3, 4, 5, 6, 7])
y = np.array([2.5, 3.2, 3.8, 6.5, 11.5])

# Fit a 2nd-degree polynomial: y = a2*x^2 + a1*x + a0
coeffs = np.polyfit(x, y, deg=2)  # Returns [a2, a1, a0]
a2, a1, a0 = coeffs

# Print the full formula without using f-string
print("y =", a2, "x² +", a1, "x +", a0)
```

```
y = 0.7642857142857119 x² + -5.512857142857122 x + 12.428571428571374
```

ID3

In [1]:
```
#iris dataset
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Load the dataset
```

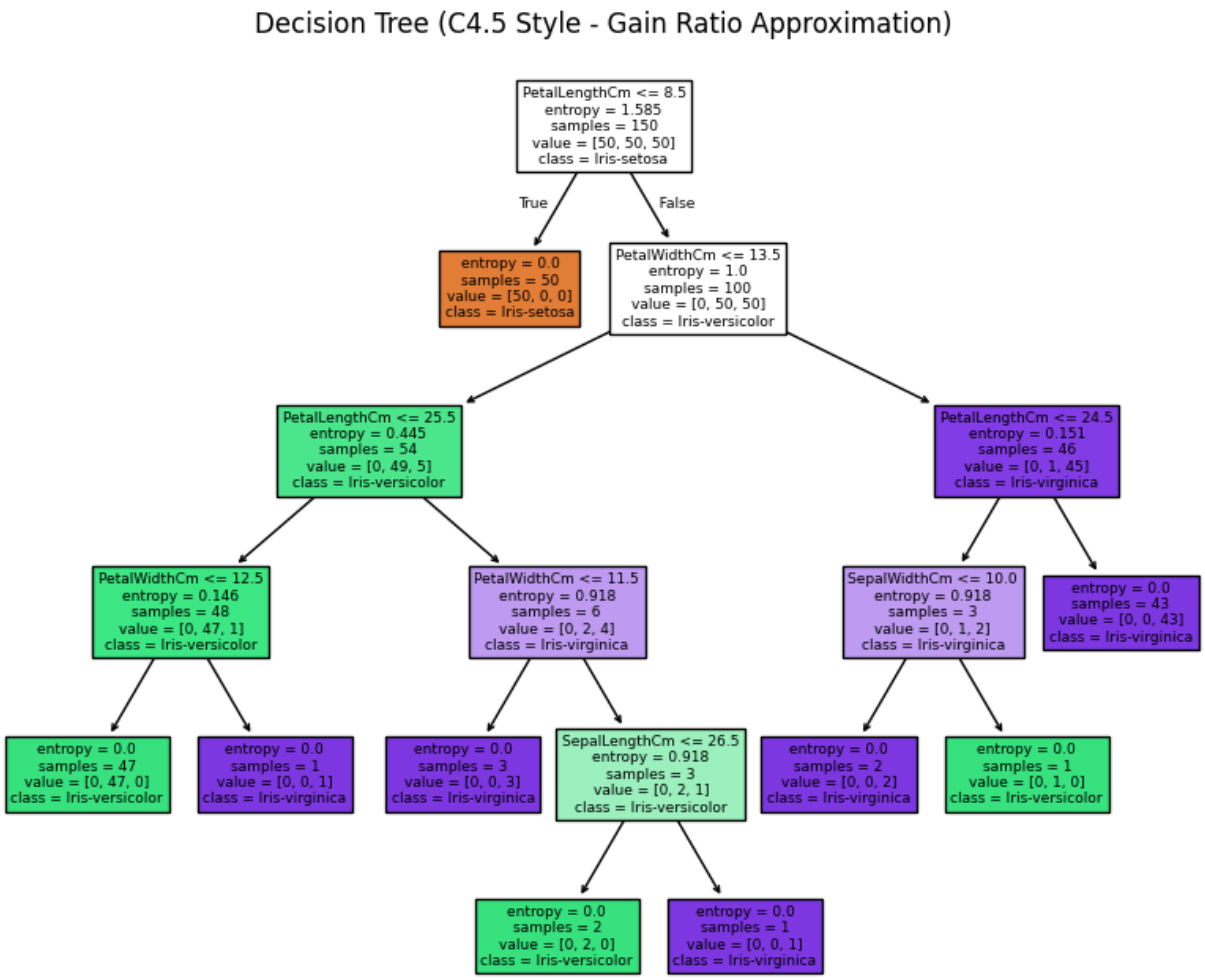
```
df = pd.read_csv(r'C:\saakshi\AI&ML_Dataset\\iris.csv')

df.drop(columns=["Id"], inplace=True)

le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

X = df.drop(columns=["Species"])
y = df["Species"]

model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

plt.figure(figsize=(10, 10))
plot_tree(model, feature_names=X.columns, class_names=["Iris-setosa", "Iris-ve
plt.title("Decision Tree (ID3 Algorithm)")
plt.show()
```



C4.5

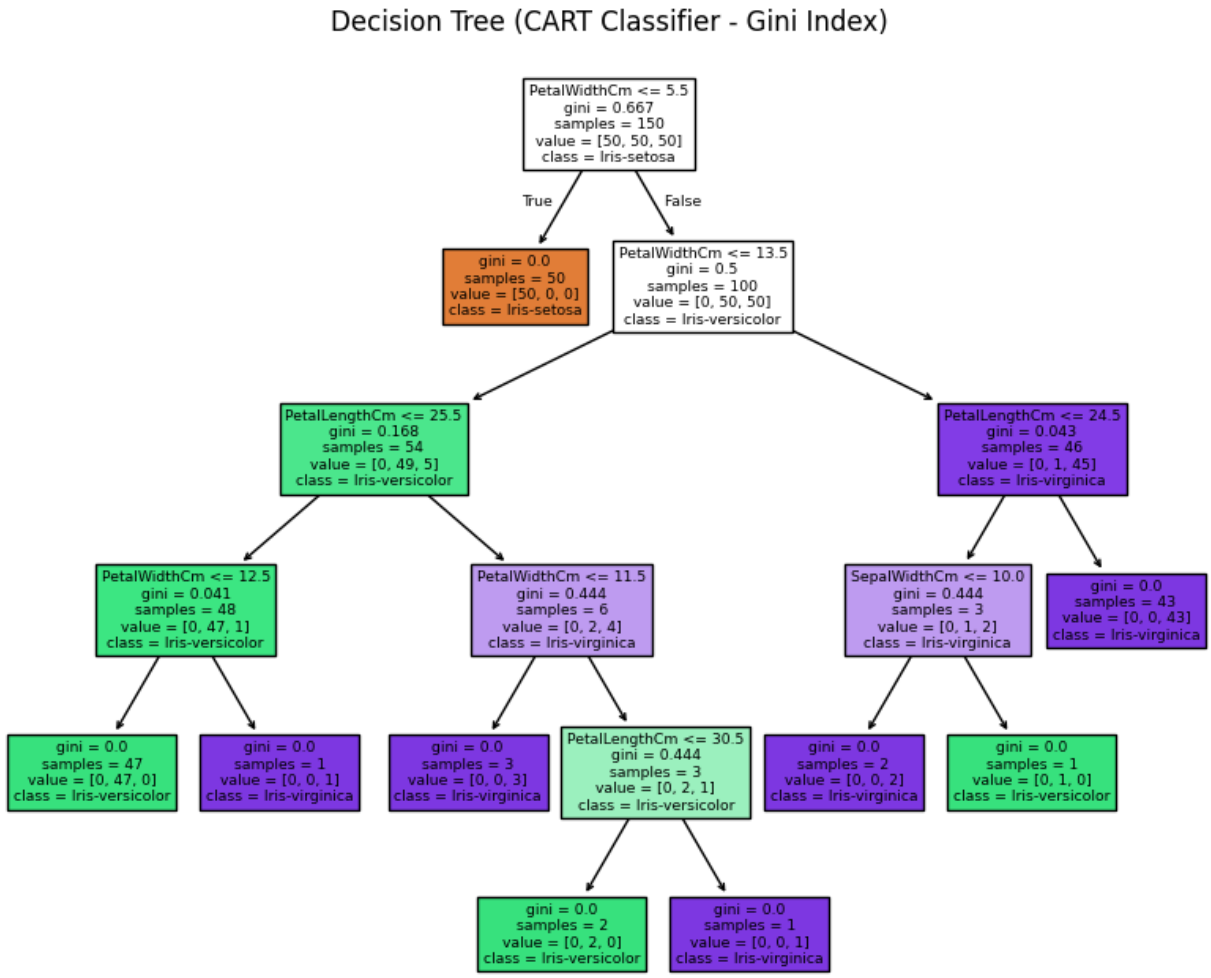
In [12]:
```
#iris dataset
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Load and preprocess the dataset
df = pd.read_csv(r'C:\saakshi\AI&ML_Dataset\iris.csv')
df.drop(columns=["Id"], inplace=True)

# Encode categorical columns
```

```
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Species"])
y = df["Species"]
)
# Train decision tree using entropy (C4.5-style
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["Iris-setosa", "Iris-ve
plt.title("Decision Tree (C4.5 Style - Gain Ratio Approximation)")
plt.show()
```



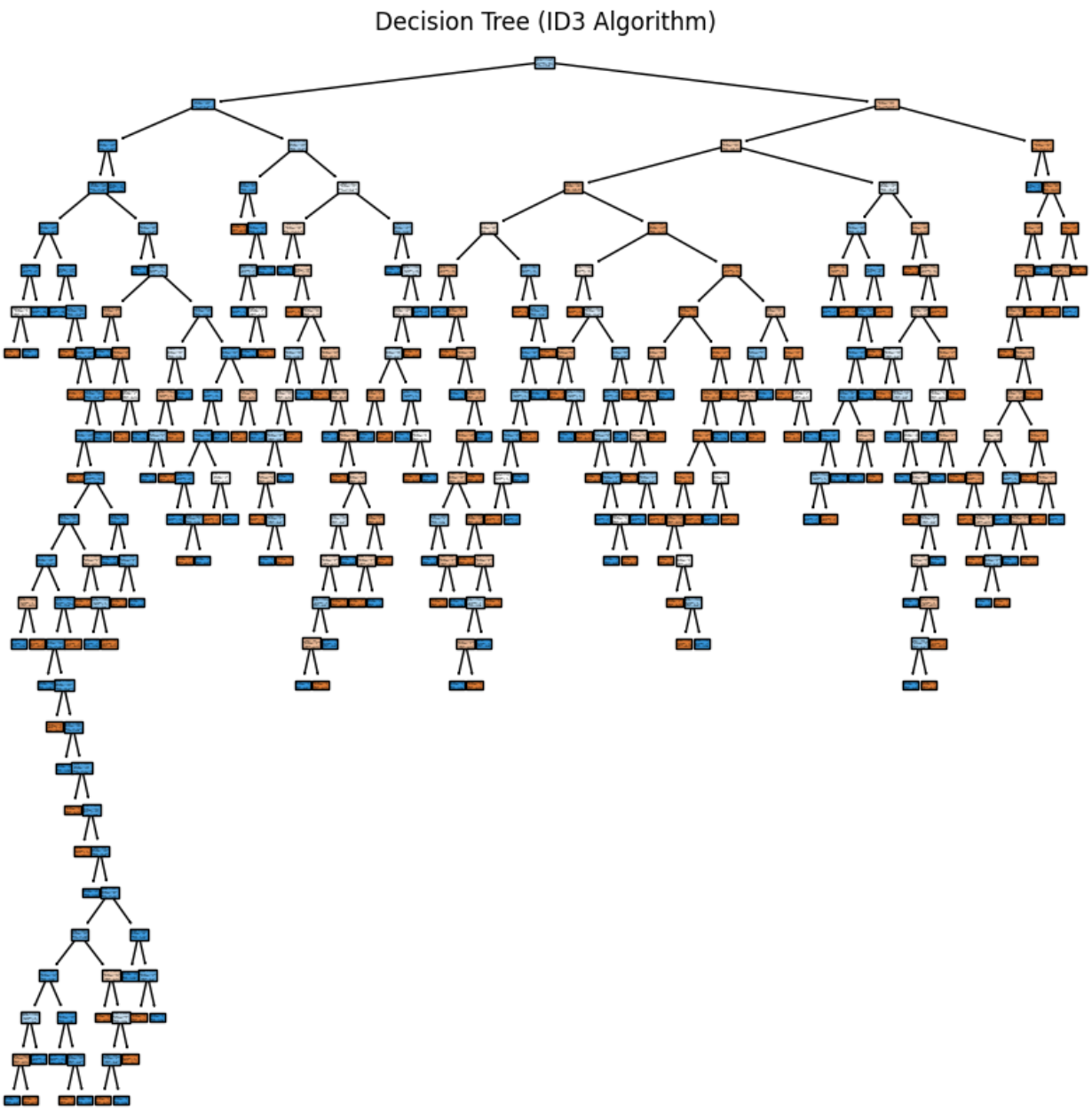
cart classifier

In [13]:
```
#iris dataset
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

df = pd.read_csv(r'C:\saakshi\AI&ML_Dataset\iris.csv')
df.drop(columns=["Id"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Species"])
y = df["Species"]

# Train decision tree using CART (Gini index)
model = DecisionTreeClassifier(criterion="gini")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["Iris-setosa", "Iris-ve
plt.title("Decision Tree (CART Classifier - Gini Index)")
plt.show()
```



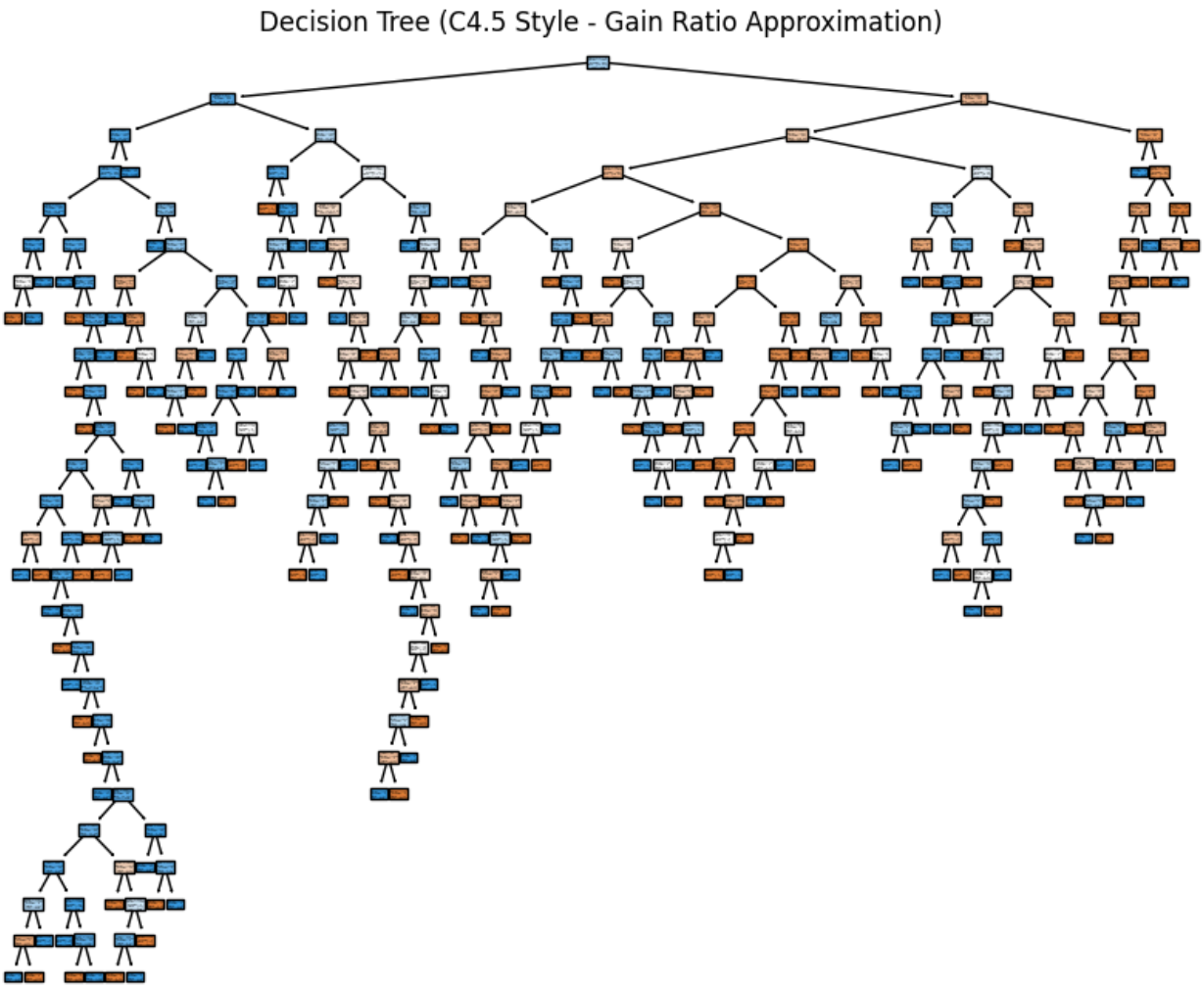
In [2]:

```
#titanic dataset ID3
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
# Load the dataset
df = pd.read_csv(r'C:\saakshi\AI&ML_Dataset\\titanic.csv')
df.drop(columns=["PassengerId"], inplace=True)
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])
X = df.drop(columns=["Sex"])
y = df["Sex"]
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)
plt.figure(figsize=(10, 10))
plot_tree(model, feature_names=X.columns, class_names=["female", "male"], fill
plt.title("Decision Tree (ID3 Algorithm)")
plt.show()
```



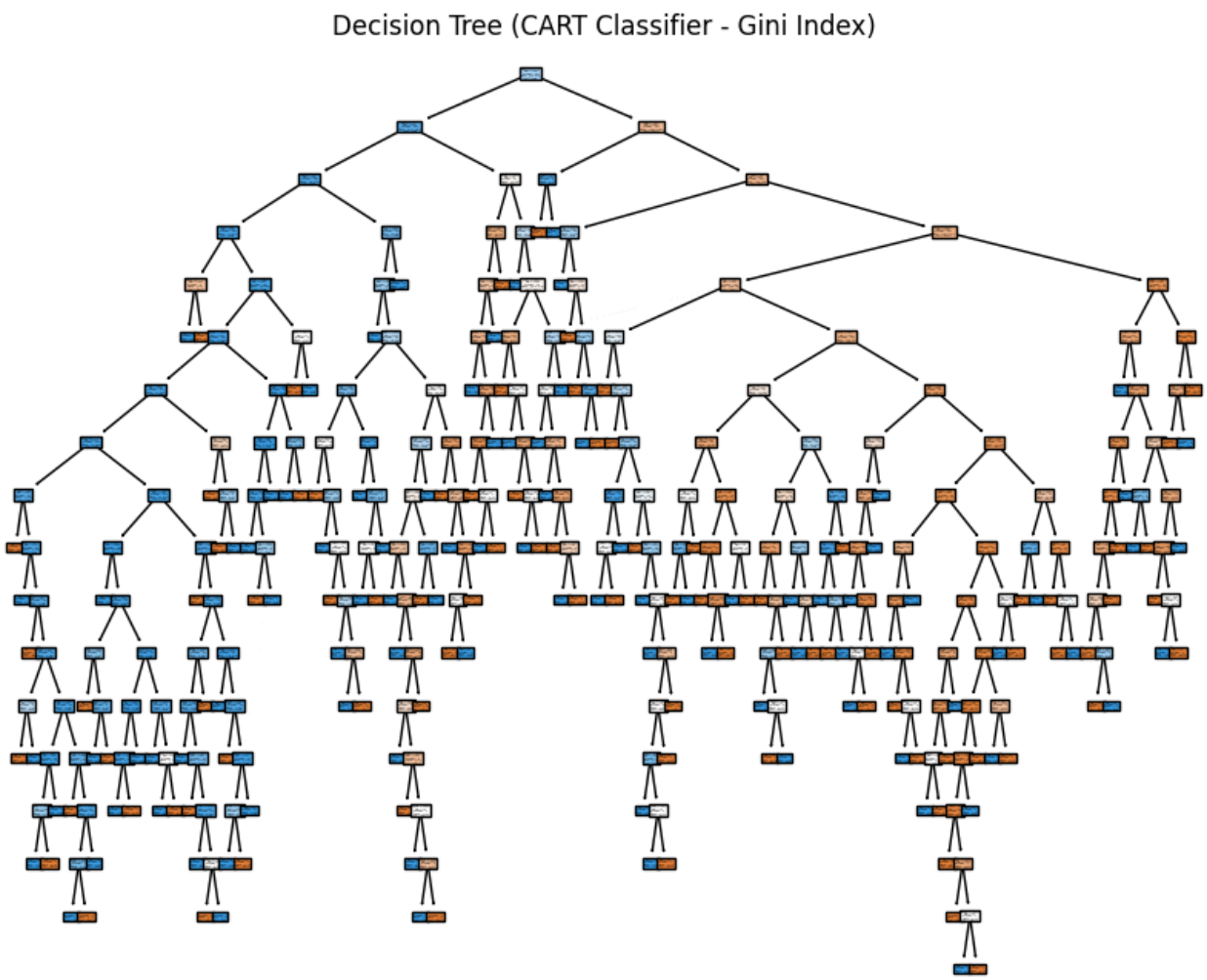
In [4]:
```
#titanic dataset c4.5
#iris dataset
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder

# Load and preprocess the dataset
df = pd.read_csv(r'C:\saakshi\AI&ML_Dataset\\titanic.csv')
df.drop(columns=["PassengerId"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])
```

```
# Split features and target
X = df.drop(columns=["Sex"])
y = df["Sex"]

# Train decision tree using entropy (C4.5-style
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["female", "male"], fill
plt.title("Decision Tree (C4.5 Style - Gain Ratio Approximation)")
plt.show()
```



In [5]:
```
#titanic dataset cart classifier
#iris dataset
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
```

```python
df = pd.read_csv(r'C:\saakshi\AI&ML_Dataset\\titanic.csv')
df.drop(columns=["PassengerId"], inplace=True)

# Encode categorical columns
le = LabelEncoder()
for column in df.columns:
    df[column] = le.fit_transform(df[column])

# Split features and target
X = df.drop(columns=["Sex"])
y = df["Sex"]

# Train decision tree using CART (Gini index)
model = DecisionTreeClassifier(criterion="gini")
model.fit(X, y)

# Visualize the tree
plt.figure(figsize=(10, 8))
plot_tree(model, feature_names=X.columns, class_names=["female", "male"], fill
plt.title("Decision Tree (CART Classifier - Gini Index)")
plt.show()
```



In [ ]: